

AI Agents

Limitations of LLMs

So far, we have experimented with LLMs, and they seem to be all knowledgeable and powerful in answering single questions and multiple related questions via chat, analyzing documents, predicting image content, etc. However, there are many useful things that an LLM cannot do on its own. For example, below are a few actions that an LLM on its own can't do:

- Create a text file at C:\Temp\test.txt with content "hello world!".
- Open Outlook and send an email to john@company.com. Tell John that I won't be able to attend the meeting next Monday due to an urgent trip I have to take to Boston.
- Connect to the Sales database and retrieve information about order number 123456.
- Contact the FedEx web service and check the delivery status of the package with tracking ID FTY8877UHY67543.
- What is today's price of Microsoft stock (symbol MSFT)?
- What is today's weather forecast for Seattle, Washington?

These are just a few examples. Because we can't do things like these, the ability to create more sophisticated workflows is limited if we just rely on the LLM by itself.

We will first demonstrate that an LLM can't do operations like the above on its own.

1. Open the LLMClient VSCode project that you worked on previously.
2. In LM Studio, ensure that the model is loaded and the server status is Running. Also, disable the **Enable Thinking** setting under the **Inference** tab in the model properties pane.
3. Open file chat_with_model2.py. Run the file by clicking the **Run Python File** button.
4. At the **You:** label, enter the prompt shown below and press the Enter key:

Create a text file at C:\Temp\test1.txt with content "hello

world!"

If you read the first few lines of the LLM response, it tells you that it can't do that. Here is what I got when I tried it:

```
You: Create a text file at C:\Temp\test1.txt with content "hello world!"
Assistant: As an AI model, I operate in a virtual environment and **do not have the ability to directly interact with your local file system** (like creating files on 'C:\Temp\'). My capabilities are limited to generating text, code, or providing instructions.
```

- Let's try another prompt that it can't do: At the **You:** label, enter the prompt shown below and press the Enter key:

What is today's price of Microsoft stock (symbol MSFT)?

If you read the first sentence of its response, it tells you that it does not have access to current market data:

```
Assistant: As an AI language model, I **do not have real-time access to live market data**.
```

- Let's try one more action that the LLM likely won't be able to do: At the **You:** label, enter the prompt shown below and press the Enter key:

What is the status of the FedEx package delivery with tracking ID FTY8877UHY67543?

Same expected reply: It can't do it.

5. Try asking it to do a few things that you think it can't do. For example, ask it if you have any unread emails in your Inbox. Or, how much storage you have left on your disk drive.

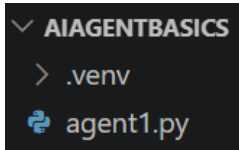
Improving capability by adding tools

6. Create a project folder and name it AI-Agent-Basics.
7. Open AI-Agent-Basics in VSCode and create a python virtual environment.

- From the command prompt in the TERMINAL pane, run the following command to install the `lmstudio` package into the project:

```
pip install lmstudio
```

- While `AIAGENTBASICS` is selected, create a new file named `agent1.py`. Your project folder structure should now look like this:



- Copy all the code of file `chat_with_model2.py` (in project `LLMClient`) and paste it into `agent1.py`.

- Modify `agent1.py` as follows:

```
# Import needed packages
import lmstudio as lms
from sys import stdout
import random

# Define some colors to make our chat interaction look pretty
GREEN = "\033[0;32m"      # User prompt starts with 'You: ' shown in green
BLUE = "\033[34m"         # Model reply starts with 'Assistant: ' shown in blue
YELLOW = "\033[33m"       # Special instruction words are shown in yellow
DEFAULT_COLOR = "\033[0m" # Reset to default color

def get_temperature(location: str):
    """Gets the temperature at the specified location."""

    # In a real case, you'd call a weather web service and get the weather information
    # at the specified location. Here we are faking it by returning a random temperature
    # value between 40 and 85

    temperature = random.randint(40, 85) # Get a random number between 40 and 85
    return f"The temperature in {location} is {temperature} oF"

def print_fragment(fragment, round_index=0):

    # .act() supplies the round index as the second parameter
    # Setting a default value means the callback is also
    # compatible with .complete() and .respond().
    print(fragment.content, end="", flush=True)

def start_new_chat():
    # Create an instance of class Chat.
    chat = lms.Chat("You are a task focused AI assistant")
    return chat
```

```

# Get a handle to the model used
model = lms.llm("google/gemma-4-e4b")

# Print an introduction sentence with instructions how to quit
print()
print("I am an assistant, and I am happy to help you. Just ask questions and we can converse.")
stdout.write(DEFAULT_COLOR + "When ready to start a new conversation, just type: " + YELLOW + "New" + DEFAULT_COLOR)
stdout.write(DEFAULT_COLOR + "\nWhen ready to terminate the program, just type: " + YELLOW + "Thank you" + DEFAULT_COLOR)

chat = start_new_chat()

while True:
    stdout.write(GREEN + "\nYou: " + DEFAULT_COLOR)
    user_input = input()
    if len(user_input) == 0:
        continue

    if user_input.lower() == "thank you":
        break

    if user_input.lower() == "new":
        chat = start_new_chat()
        continue

    # Add the user input to the chat history
    chat.add_user_message(user_input)
    stdout.write(BLUE + "Assistant: " + DEFAULT_COLOR)

    # Request a response with implicit tool use in an ongoing chat session
    # In this case we defined one tool: the get_weather function
    model.act(
        chat,
        [get_temperature],
        on_message=chat.append,
        on_prediction_fragment=print_fragment,
    )

```

12. Save the changes you made and run the `agen1.py` file by clicking the **Run Python File** button.

13. Ask the model a generic question: *“What is the capital of Mexico?”*

Then ask it this question: *“What is the current temperature in Seattle?”*

Then ask it this question: *“What is current market value of Amazon stock (symbol AMZN)?”*

Then ask it another generic question: *“Are dolphins mammals?”*

Notice that it was able to get you a temperature answer but not a market value of a company stock.

```

You: What is the current temperature in Seattle?
Assistant: The temperature in Seattle is 50 oF.
You: What is the current price of Amazon stock (symbol AMZN)?
Assistant: I am sorry, but I do not have the capability to provide real-time stock prices. You would need to check a financial news website or a dedicated stock market application for the current price of Amazon (AMZN).

```

The reason it was able to answer the temperature in Seattle question is because we created a tool (function `get_temperature()`) and passed that tool in the `model.act()` function:

```
model.act(  
    chat,  
    [get_temperature],  
    on_message=chat.append,  
    on_prediction_fragment=print_fragment,  
)
```

The documentation of the `model.act()` function says:

“Request a response (with implicit tool use) in an ongoing agent chat session.”

We refer to `get_temperature` in this case as a **tool that we give the LLM to use**. And this tool allows it to do something that it can’t do on its own. In a way, we augmented its capabilities with some new operation that it can execute and get back an answer. Tools are just functions in python.

Because we did not provide it with a stock value tool, it wasn’t able to get a market value. You will now modify the code to give it a stock value tool.

14. Add the following function immediately after the `get_temperature` function:

```
def get_stock_market_price(symbol: str):  
    """Gets the stock market price of the specified stock symbol"""  
  
    # In a real case, you'd call a financial web service such as Bloomberg or the New York  
    # Stock Exchange and get a true market value. In this case we are faking it by making  
    # a number between 100 and 500  
    current_price = random.randint(100, 500)  
    return f"The current market value of {symbol} is ${current_price}"
```

And modify the `model.act()` function to add this new `get_stock_market_price` tool.

```
model.act(  
    chat,  
    [get_temperature, get_stock_market_price],  
    on_message=chat.append,  
    on_prediction_fragment=print_fragment,  
)
```

Tools are passed to `model.act()` in the second argument as a python list: you just add the function names separated by commas in a list: [...]

15. Save the changes you made to `agent1.py` (menu **File** → **Save**) and run the code by clicking the **Run Python File** button.

Ask the same four questions you asked before. It should now give answers to all four questions.

“What is the capital of Mexico?”

“What is the current temperature in Seattle?”

“What is current market value of Amazon stock (symbol AMZN)?”

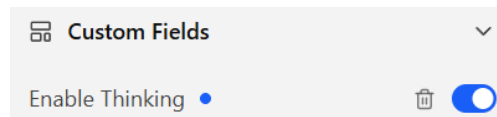
“Are dolphins mammals?”

The questions in green are generic informational questions. It learned to answer them when it was trained. The questions in blue require access to specific live data and it can only answer them if we augment its capabilities with tools. In this case we defined two tools:

```
get_temperature  
get_stock_market_price
```

And we used the `model.act()` function to tell the LLM to use them when needed.

16. Type Thank you to terminate the chat. Then go to LM Studio → Developer page → Local Server and turn on Enable Thinking:



Run `agen1.py` and observe the thinking when you ask the temperature or stock questions:

```

You: What is the current price of Microsoft stock (symbol MSFT)?
Assistant: 1. Analyze the Request: The user is asking for the "current price" of "Microsoft stock" with the symbol "(symbol MSFT)".

2. Examine Available Tools:
   * get_temperature(location: STRING): Gets temperature. Irrelevant to stock prices.
   * get_stock_market_price(symbol: STRING): Gets the stock market price of a specified stock symbol. This is relevant.

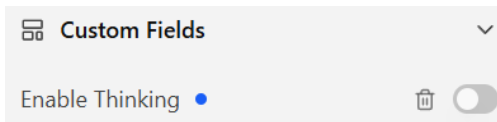
3. Determine Tool Usage: The appropriate tool is get_stock_market_price. The required parameter is symbol, and the user provided "MSFT".

4. Construct the Function Call: call:get_stock_market_price(symbol:"MSFT")

5. Final Output Generation: Format the function call into the required JSON format for tool use. __LM_STUDIO_INTERNAL_LSEP_SYNTHETIC_REASONING_END_f4e9a8d2c6b14d0c9e5f3a7b8c1d2e6a__ The current market value of MSFT is $255.

```

17. Type Thank you to terminate the chat. Then go to LM Studio → Developer page → Local Server and turn off Enable Thinking:



What is an AI Agent?

Definition from IBM:

“An artificial intelligence agent is a system that autonomously performs tasks by designing workflows with available tools.”

Go to this IBM [site](#) and read the paragraph **Reasoning with available tools**.

Exercises to do:

Exercise 1

Create a tool that creates a file on your PC. You can name the function `create_file`. The function should take two parameters: `file_path` which is the path of the file to create, and `content` which is the data to write to the file. Both `file_path` and `content` are of type string.

Then test your tool by asking the LLM to create a file for you. For example:

You: Create a file at C:\Temp\test_file.txt with content Bellevue College

Then verify that test_file.txt was created under C:\Temp and that it contains the string Bellevue College. Note that you need to have folder Temp under C for this example to work

Exercise 2

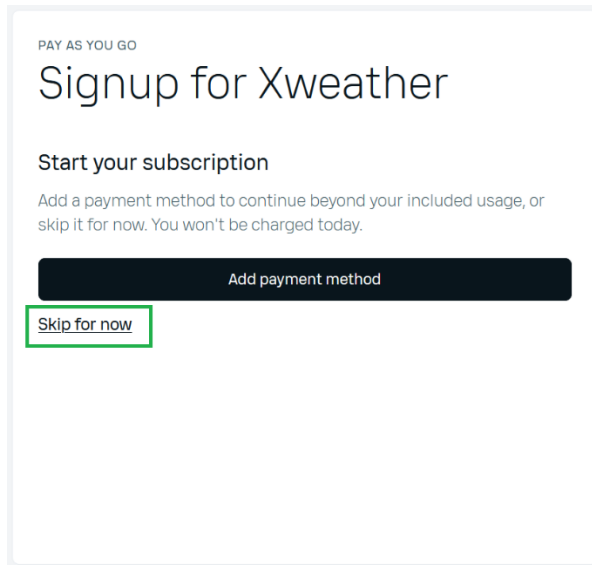
Create a tool that cleans a directory of certain files. For example, assume you have a folder with 20 files in it, and you want to delete all files with a given extension (e.g., .jpg files or files that have the word 'tmp' in their name). You can name your tool delete_files. The function should take two parameters: folder_path which is the path of the folder to search, and pattern which is any string that exists in the file name. The tool should delete all files with the pattern in their names. For example, assume I have a folder (C:\Temp) with these files in it:

data1.tmp
data2.tmp
purchase.pdf
birthday_pic.jpg
tmp_english_paper.docx
test.txt
data.xml

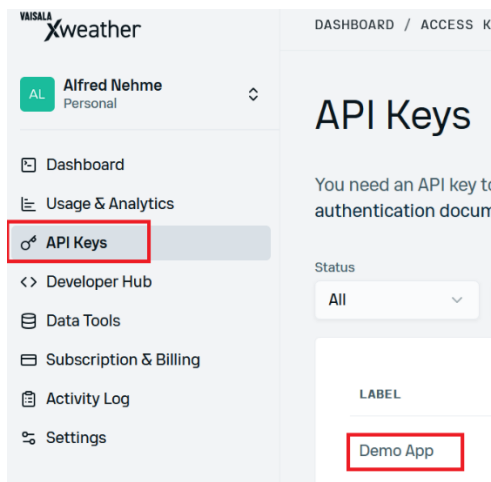
If you tell the LLM to delete all files in C:\Temp that have tmp in their names, then it should delete files data1.tmp, data2.tmp and tmp_english_paper.docx.

Exercise 3

- Go to <https://www.xweather.com/products/weather-api>
- Click the **Get your free API key** button.
- Click the **Sign up for free** button.
- Choose the **Personal Use** option, fill out the form (you need to enter a valid email that you can check on your mobile or PC), and click the **Submit** button.
- When you get to this screen, just click the **Skip for now** link:



- On the next page, you should see your API key.
- Wait a minute or two until you receive a second email from Xweather where they ask you to login (click the **Login to your dashboard** button in the email and login using the email and password you chose when you created an account). This takes you to your dashboard page.
- In your Dashboard page, click the **API Keys** link (from the left navigation bar) followed by the **Demo App** link.



- This takes you to a page where you will see your **Key**, **Client ID**, and **Secret**. You need those to interact with the xweather service.

I suggest you create a file on your desktop (name it `weather_keys.txt`), and copy and paste into it your Key, Client ID, and Secret.

Testing the weather service:

- In the AI-Agent-Basics VSCode project, create a new file and name it `test_weather_api.py`.

Write code (based on the python example code they give you at the bottom of [this page](#)) to test that you can interact with the weather API.

Once you have a successful test, move the code from `test_weather_api.py` to function `get_temperature` in `agent1.py`.

You should now have a tool function that gets real live temperature values. Test it with your LLM

Exercise 4

Get live market values of stocks using an API from <https://www.alphavantage.co/>: Sign up, get a free API key, and do something similar to how you did it for weather (in this case you need to replace the code in function `get_stock_market_price`). You need to at their website or do a bit of research to figure out how to call their API from python.

You should now have a tool function that gets real current market values of stocks. Test it with your LLM